LAPORAN PROJECT UTS

SMART CONTRACT–DRIVEN DECENTRALIZED TAX SYSTEM: TRANSFORMASI TRANSPARANSI PAJAK PUSAT DENGAN TEKNOLOGI BLOCKCHAIN

INSTITUT TEKNOLOGI PLN

Disusun oleh:

Tara Imelda Fatin	**(202332164)**
Citra Delvina Zahrin	**(202332167)**
Ahmad Rajif	**(202332168)**
Amelya Putri	**(202332169)**

FAKULTAS TELEMATIKA ENERGI

PROGRAM STUDI SISTEM INFORMASI

INSTITUT TEKNOLOGI PLN

JAKARTA BARAT

TAHUN 2026

ABSTRAK

Sistem perpajakan pusat di Indonesia menghadapi tantangan fundamental berupa defisit transparansi, inefisiensi proses restitusi, dan fragmentasi data antar instansi. Penelitian ini merancang sistem pembayaran dan transparansi pajak berbasis blockchain Ethereum menggunakan smart contract untuk menciptakan ekosistem perpajakan yang transparan, akuntabel, dan efisien. Metode yang digunakan adalah analisis sistem terdistribusi dengan pendekatan Byzantine Fault Tolerance untuk mengidentifikasi vulnerabilitas dalam sistem konvensional.Solusi yang diusulkan berupa consortium blockchain dengan smart contract yang mengimplementasikan lima fungsi utama: pencatatan pembayaran (recordPayment), pengajuan restitusi (requestRestitution), persetujuan multi-signature (approveRestitution), verifikasi publik (verifyPayment), dan pelacakan alokasi dana (trackAllocation).

Sistem ini menggunakan mekanisme konsensus Proof of Authority (PoA) pada testnet Sepolia dengan integrasi oracle untuk validasi data eksternal. Hasil rancangan menunjukkan bahwa implementasi blockchain dapat mengurangi delay settlement dari 1-3 hari menjadi real-time, mencegah double spending melalui model UTPO (Unspent Tax Payment Output), dan menyediakan immutable audit trail untuk akuntabilitas publik. Smart contract dikembangkan menggunakan Solidity 0.8.x dengan fitur access control berbasis role dan event logging untuk transparansi. Penelitian ini memberikan kontribusi konseptual dalam transformasi digital sistem perpajakan Indonesia melalui teknologi blockchain yang terdesentralisasi namun tetap terkontrol.

Kata Kunci: Blockchain, Smart Contract, Transparansi Pajak, Ethereum, Restitusi

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Sistem pembayaran dan transparansi pajak pusat di Indonesia merupakan ekosistem kompleks yang melibatkan multi-stakeholder termasuk Direktorat Jenderal Pajak (DJP), Kantor Pelayanan Pajak (KPP), Wajib Pajak, Bank Persepsi, Bea Cukai, dan lembaga pengawas seperti BPK dan KPK. Namun, sistem yang berjalan saat ini menghadapi tiga masalah fundamental yang saling berkaitan.

Pertama, defisit transparansi dalam alokasi dana pajak. Wajib Pajak tidak memiliki mekanisme untuk melacak bagaimana kontribusi pajak mereka dialokasikan ke program-program pemerintah. Mekanisme pelaporan bersifat agregat, periodik, dan sulit diakses masyarakat umum, menimbulkan persepsi "pajak sebagai iuran paksa" tanpa imbal balik yang jelas.

Kedua, inefisiensi dan kerentanan dalam proses restitusi. Proses pengembalian kelebihan pembayaran pajak melibatkan 7-10 tahap verifikasi manual yang memakan waktu 3-12 bulan. Kompleksitas prosedur ini menciptakan celah untuk manipulasi dokumen, penundaan disengaja, dan ketidakpastian bagi Wajib Pajak dalam perencanaan arus kas.

Ketiga, fragmentasi data antar institusi pemerintah. Data Wajib Pajak tersebar di berbagai sistem yang tidak terintegrasi sempurna seperti DJP, Bea Cukai, Ditjen AHU, Bank Persepsi, dan PPATK. Ketidaksinkronan data ini menghambat validasi real-time dan meningkatkan risiko kesalahan administrasi serta potensi double claim atau penolakan yang tidak konsisten.

Teknologi blockchain menawarkan solusi inovatif untuk mengatasi permasalahan-permasalahan ini melalui karakteristik distributed ledger, immutability, dan smart contract. Dengan implementasi yang tepat, blockchain dapat menciptakan single source of truth terdistribusi, menyediakan audit trail yang tidak dapat dimanipulasi, dan mengotomatisasi proses bisnis perpajakan melalui smart contract.

1.2 Rumusan Masalah

1. Bagaimana merancang arsitektur blockchain permissioned yang mengintegrasikan multi-stakeholder perpajakan tanpa mengorbankan keamanan data?

2. Bagaimana mengimplementasikan smart contract untuk mengotomatisasi alur restitusi dengan mekanisme persetujuan terdesentralisasi?

3. Bagaimana mekanisme blockchain mencegah *double spending* dan menyediakan pelacakan alokasi dana yang dapat diaudit publik?

1.3 Tujuan

1. Merancang arsitektur sistem berbasis Ethereum/Sepolia dengan konsensus PoA untuk lingkungan enterprise perpajakan.

2. Mengembangkan skeleton smart contract Solidity yang mencakup fungsi pencatatan, pengajuan, approval, dan verifikasi.

3. Menyediakan rancangan antarmuka dan alur data yang mendukung transparansi publik serta kepatuhan regulasi.

1.4 Ruang Lingkup

Penelitian ini membatasi pembahasan pada perancangan smart contract Solidity 0.8.x di testnet Sepolia dengan lima fungsi inti perpajakan dan mekanisme RBAC, serta simulasi oracle untuk validasi data eksternal. Aspek yang tidak dibahas meliputi migrasi sistem Coretax, audit keamanan produksi, dan implikasi regulasi hukum. Fokus penelitian adalah prototipe teknis untuk transparansi pembayaran dan restitusi pajak menggunakan Ethereum Virtual Machine, Ethers.js, dan Remix IDE, tanpa pengujian beban skala nasional atau implementasi front-end produksi. Validasi prototipe dilakukan melalui pengujian fungsional pada lingkungan testnet untuk memastikan kesesuaian logika bisnis dengan spesifikasi administrasi perpajakan. Pembatasan ruang lingkup ini diterapkan secara strategis agar analisis dapat mendalam pada aspek teknis blockchain tanpa mengorbankan fokus pada solusi transparansi dan akuntabilitas. Hasil penelitian diharapkan menjadi referensi konseptual bagi pengembangan sistem perpajakan digital yang mengintegrasikan prinsip desentralisasi blockchain dengan kerangka regulasi fiskal nasional.

1.5 Kontribusi Anggota Kelompok

Penelitian ini merupakan hasil kerja kolaboratif dengan pembagian peran sebagai berikut:

Ahmad Rajif (Ketua Kelompok / Smart Contract Developer) Bertanggung jawab atas perancangan dan implementasi smart contract TaxTransparency.sol, termasuk lima fungsi inti perpajakan, mekanisme RBAC, dan approval multi-signature. Melakukan deployment dan pengujian kontrak pada testnet Sepolia menggunakan Remix IDE serta memastikan kepatuhan kode terhadap standar Solidity 0.8.x.

Tara Imelda Fatin (System Architect & Data Flow Designer) Merancang arsitektur sistem 3-layer dan memodelkan alur pertukaran data antar komponen. Menyusun spesifikasi validasi data eksternal (Dukcapil/Bea Cukai) dalam bentuk dokumen desain API, serta membuat diagram arsitektur dan integrasi sistem menggunakan notasi standar untuk memandu implementasi teknis.

Amelya Putri (Front-End Developer & UI/UX Designer) Mengembangkan prototipe antarmuka Portal Wajib Pajak dan Dashboard Publik, termasuk integrasi MetaMask menggunakan Ethers.js. Merancang visualisasi alokasi dana yang intuitif serta memastikan responsivitas dan aksesibilitas antarmuka.

Citra Delvina Zahrin (Security Architect & Consensus Mechanism Designer) Merancang konfigurasi jaringan consortium blockchain dengan mekanisme konsensus Proof of Authority (PoA) dan menentukan spesifikasi validator node (DJP, Bank, BPK). Menganalisis aspek keamanan arsitektur seperti pencegahan serangan Byzantine, manajemen kunci kriptografis, dan strategi isolasi channel privat untuk melindungi data sensitif Wajib Pajak.

BAB 2

TINJAUAN PUSTAKA

2.1 Konsep Blockchain

Blockchain adalah teknologi *distributed ledger* yang mencatat transaksi secara terdesentralisasi, transparan, dan tidak dapat diubah (*immutable*). Diperkenalkan oleh Nakamoto (2008) sebagai dasar Bitcoin, setiap blok berisi transaksi tervalidasi, timestamp, dan hash dari blok sebelumnya. Sifat *immutable* dijaga dengan kriptografi seperti SHA-256, sehingga data sulit dimanipulasi tanpa persetujuan jaringan.

2.2 Kriptografi: Hash dan Digital Signature

Kriptografi adalah dasar keamanan blockchain. Fungsi hash seperti keccak256 (SHA-3) menghasilkan identitas unik dengan sifat deterministic, cepat, dan tahan terhadap collision.

Digital signature menggunakan ECDSA untuk memverifikasi transaksi. Transaksi ditandatangani dengan *private key* dan diverifikasi dengan *public key*, sehingga menjamin keaslian, integritas, dan *non-repudiation*.

2.3 Mekanisme Konsensus

Mekanisme konsensus adalah protokol yang memungkinkan node dalam jaringan blockchain mencapai kesepakatan mengenai validitas transaksi tanpa otoritas pusat.

Proof of Work (PoW) mengharuskan miner menyelesaikan puzzle kriptografis sebelum menambahkan blok baru; mekanisme ini aman namun memiliki konsumsi energi tinggi dan throughput rendah. Sementara itu, **Proof of Stake (PoS)** memilih validator berdasarkan jumlah aset yang di-*stake*, sehingga lebih efisien energi dan mampu memproses transaksi lebih cepat.

Untuk kebutuhan enterprise, mekanisme seperti **Proof of Authority (PoA)** atau Practical Byzantine Fault Tolerance (PBFT) lebih sesuai karena memberikan finalitas transaksi yang cepat, throughput tinggi, serta kontrol akses yang lebih ketat dalam jaringan *permissioned*.

Referensi:

1. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*.
2. Buterin, V. (2014). *Ethereum Whitepaper*.
3. Allessie, M. et al. (2017). *Blockchain for Social Good*. World Bank.
4. Pratama, A. & Suryani, E. (2022). *Implementasi Blockchain untuk Transparansi Perpajakan*. J. Sistem Informasi Indonesia.
5. Menezes, A. et al. (1996). *Handbook of Applied Cryptography*. CRC Press.

BAB 3

ANALISIS STUDI KASUS

3.1 Deskripsi Studi Kasus Utama

Sistem pembayaran dan transparansi pajak pusat di Indonesia saat ini melibatkan ekosistem terdistribusi dengan alur kerja sebagai berikut:

1. Wajib Pajak melakukan pembayaran melalui Bank Persepsi atau e-Billing
2. Bank memproses dan melaporkan ke DJP dengan delay 1-3 hari kerja
3. DJP melakukan rekonsiliasi manual dengan sistem Coretax
4. Untuk restitusi, proses melibatkan 7-10 tahap verifikasi manual
5. Data tersimpan terfragmentasi di berbagai sistem instansi

3.2 Identifikasi Masalah

Berdasarkan analisis sistem existing, teridentifikasi empat masalah kritis: (1) tidak adanya mekanisme real-time tracking menyebabkan Wajib Pajak tidak dapat memverifikasi status pembayaran dan alokasi dana secara langsung, menciptakan ketidakpastian akibat delay settlement 1-3 hari; (2) proses restitusi yang panjang dan manual rentan terhadap manipulasi dokumen, penundaan selektif, serta inkonsistensi keputusan antar verifier; (3) fragmentasi data antar sistem DJP, OSS, Bea Cukai, dan bank menyebabkan inkonsistensi informasi yang berisiko memicu double claim atau penolakan tidak adil; dan (4) ketiadaan immutable audit trail menyulitkan verifikasi akuntabilitas transaksi dan keputusan administratif secara independen.

3.3 Justifikasi Blockchain vs Database Biasa

Aspek	Database Relasional (SQL)	Blockchain (Ethereum)
Otoritas	Terpusat (Admin/DJP)	Terdistribusi (Multi-validator)
Integritas	Dapat diubah admin	Immutable (hash chaining)
Trust Model	Trust-based	Trustless (konsensus kriptografis)
Double Spending	Tidak ada mekanisme native	Dicegah via UTXO/State consensus
Audit	Log terpisah, rentan edit	Terintegrasi, terbuka, terverifikasi

3.3 Studi Banding Singkat

Beberapa negara telah mengimplementasikan blockchain untuk sistem perpajakan:

Estonia: Mengimplementasikan X-Road blockchain untuk integrasi data pajak dan Bea Cukai, mengurangi waktu pemrosesan dari minggu menjadi jam.

Uni Emirat Arab: Dubai Blockchain Strategy mengintegrasikan sistem pajak dengan blockchain untuk real-time reporting dan compliance.

China: Beijing Taxation Bureau menggunakan blockchain untuk e-faktur pajak, mencegah fraud dan meningkatkan efisiensi.

Indonesia dapat belajar dari implementasi ini dengan mengadaptasi best practices ke konteks lokal.

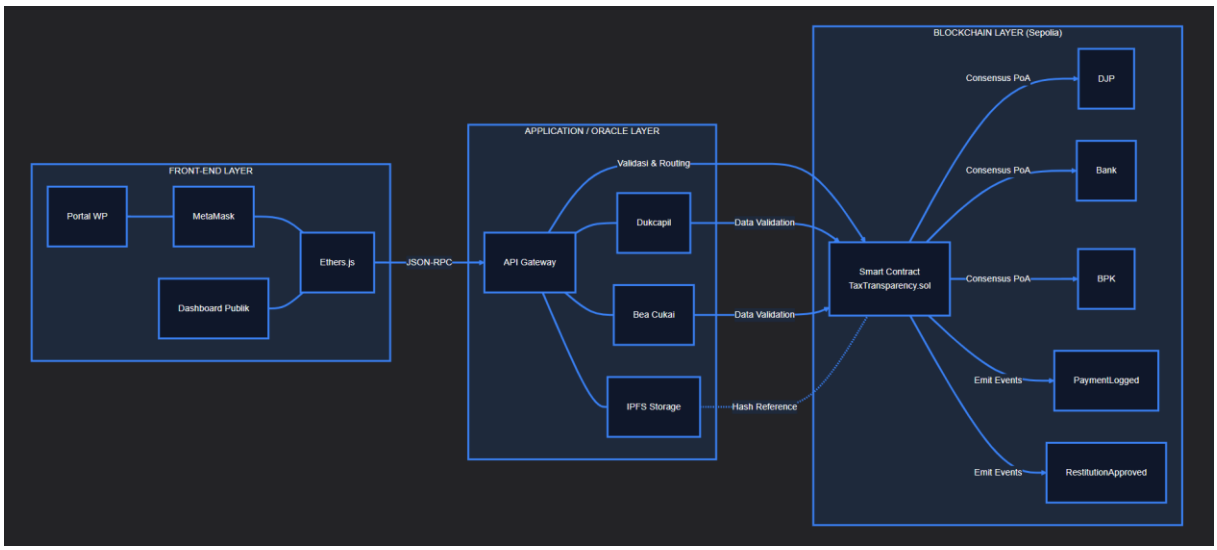
BAB 4

RANCANGAN SISTEM

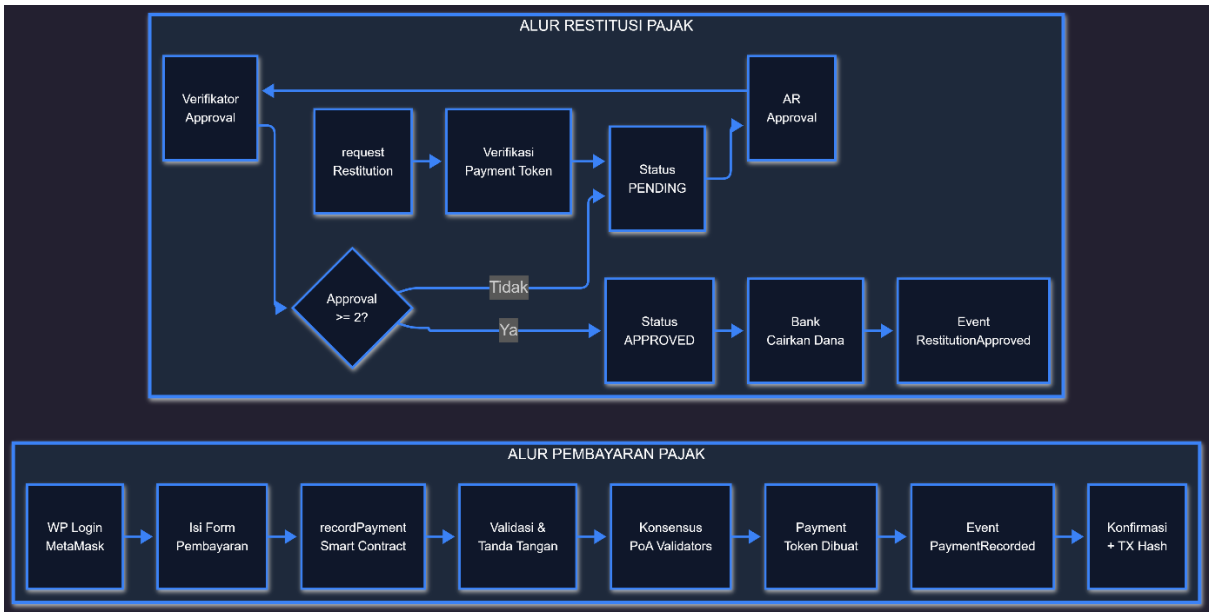
4.1 Diagram Arsitektur Sistem

Sistem terdiri dari 3 lapisan:

1. **Front-end**: Portal WP & Dashboard Publik (HTML/JS + Ethers.js + MetaMask).
2. **Application/Oracle**: API Gateway, validasi data eksternal (Dukcapil/Bea Cukai), penyimpanan dokumen off-chain (IPFS hash).
3. **Blockchain Layer**: Smart contract **TaxTransparency.sol** di Sepolia testnet, konsensus PoA, node validator (DJP, Bank, BPK).



4.2 Alur Data



Alur Pembayaran: WP connect MetaMask → isi form → call recordPayment() → sign tx → consensus → Payment Token dibuat → event emitted → status real-time.

Alur Restitusi: WP call requestRestitution() → validasi kepemilikan token → notifikasi ke AR/Verifier → approveRestitution() (min 2/3 approval) → status APPROVED → bank cairkan dana → event logged.

4.3 Skeleton Daftar Fungsi Smart Contract

```
contract TaxTransparency {
    // 1. Fungsi Pencatatan
    function recordPayment(string memory _npwp, uint256 _amt, string memory _code) public onlyWP {
        payCount++;
        payments[payCount] = Payment(payCount, _npwp, _amt, _code, msg.sender, Status.POSTED);
        emit PaymentLogged(payCount, msg.sender, _amt);
    }

    // 2. Fungsi Pengajuan
    function requestRestitution(uint256 _payId, uint256 _amt) public onlyWP returns (uint256) {
        require(payments[_payId].owner == msg.sender && _amt <= payments[_payId].amount);
        reqCount++;
        requests[reqCount] = Request(reqCount, _payId, _amt, 0, Status.PENDING);
        return reqCount;
    }

    // 3. Fungsi Approval (Multi-sig 2-of-3)
    function approveRequest(uint256 _reqId) public onlyVerifier {
        Request storage r = requests[_reqId];
        require(r.status == Status.PENDING);
        r.approvals++;
        if (r.approvals >= 2) r.status = Status.APPROVED;
        if (r.status == Status.APPROVED) emit RestitutionApproved(_reqId);
    }

    // 4. Fungsi Verifikasi Publik (Read)
    function verifyPayment(uint256 _id) public view returns (string memory, uint256, Status) {
        Payment memory p = payments[_id];
        return (p.npwp, p.amount, p.status);
    }

    // 5. Fungsi Pelacakan Alokasi (Read)
    function trackAllocation(string memory _code) public pure returns (string memory) {
        return string(abi.encodePacked("Dana dialokasikan ke program: ", _code));
    }
}
```

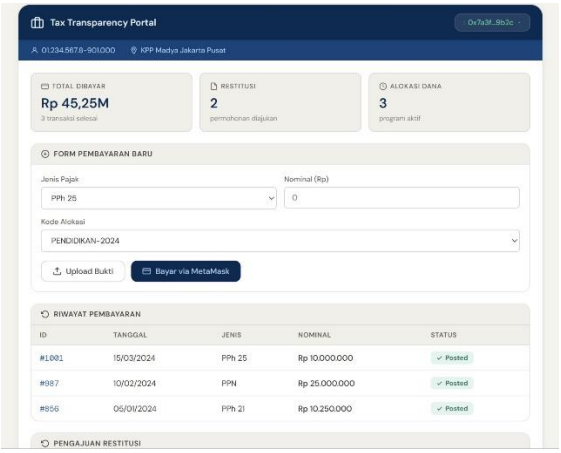
Struktur Penyimpanan Data dan Kontrol Akses On-Chain

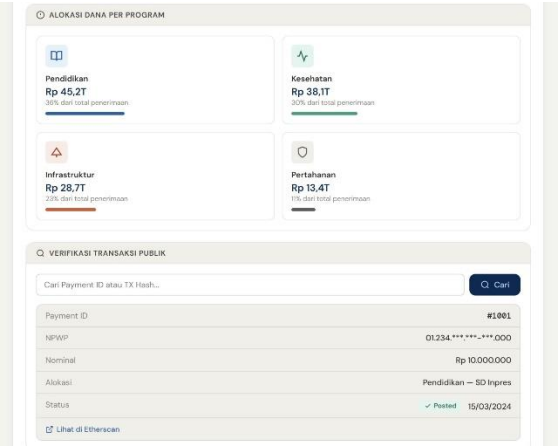
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract TaxTransparency {
    // Definisi peran pengguna untuk kontrol akses berbasis RBAC
    enum Role { WP, AR, VERIFIER, KPP_HEAD, BANK }
    // Status siklus hidup transaksi pembayaran dan restitusi
    enum Status { PENDING, POSTED, APPROVED, REJECTED }

    // Struktur data untuk mencatat pembayaran pajak
    struct Payment { uint256 id; string npwp; uint256 amount; string allocCode; address owner; Status status; }
    // Struktur data untuk mencatat permohonan restitusi dengan tracking approval
    struct Request { uint256 id; uint256 payId; uint256 amount; uint8 approvals; Status status; }

    // Mapping penyimpanan data utama: ID → Payment/Request, Address → Role
    mapping(uint256 => Payment) public payments;
    mapping(uint256 => Request) public requests;
    mapping(address => Role) public roles;
    // Counter untuk menghasilkan ID unik secara otomatis (auto-increment)
    uint256 public payCount; uint256 public reqCount;
```

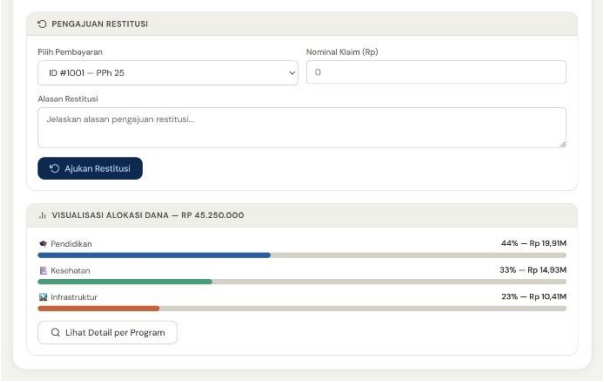
Mekanisme Transparansi Audit dan Kontrol Akses Berbasis Peran

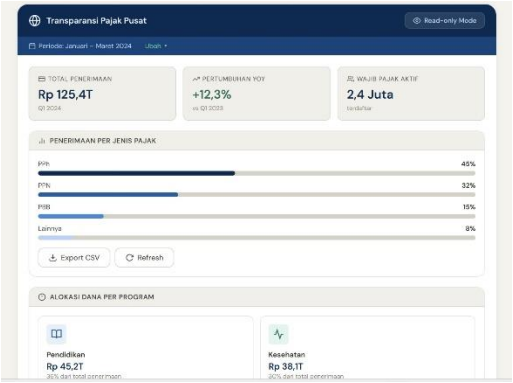
```
File  Edit  Selection  View  Go  Run  Terminal  Help        Teknologi Blockchain
Skeleton Kelompok 10.sol ×
Skeleton Kelompok 10.sol > ...
4   contract TaxTransparency {
22      // Event untuk logging transparan yang dapat di-index oleh front-end dan explorer
23      event PaymentLogged(uint256 id, address wp, uint256 amt);
24      event RestitutionApproved(uint256 reqId);
25
26      // Modifier: Membatasi akses fungsi hanya untuk pengguna dengan role WP
27      modifier onlyWP() { require(roles[msg.sender] == Role.WP); _; }
28      // Modifier: Membatasi akses fungsi hanya untuk verifier (AR/Verifikator/Kepala KPP)
29      modifier onlyVerifier() { require(roles[msg.sender] == Role.AR || roles[msg.sender] == Role.VERIFIER || roles[msg.sender] == Role.KPP_HEAD); _; }
30
```

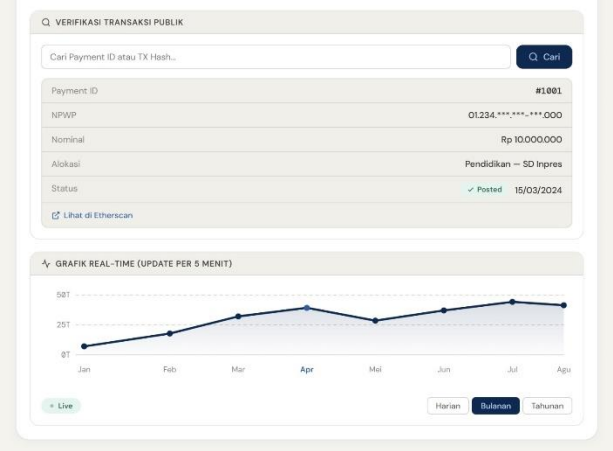
4.6 Sketsa Front-End











Lampiran

Kode Sumber Smart Contract **TaxTransparency.sol**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract TaxTransparency {
    enum Role { WP, AR, VERIFIER, KPP_HEAD, BANK, AUDITOR }
    enum PaymentStatus { PENDING, POSTED, ALLOCATED, RESTITUTED }
    enum RestitutionStatus { PENDING, APPROVED, REJECTED, DISBURSED }

    struct Payment {
        uint256 paymentId;
        string npwp;
        string taxType;
        uint256 amount;
        uint256 timestamp;
        address wpAddress;
        string documentHash;
        string allocationCode;
        PaymentStatus status;
    }

    struct RestitutionRequest {
        uint256 requestId;
        uint256 paymentId;
        uint256 amount;
        address requester;
        uint256 requestDate;
        mapping(uint8 => bool) approvals;
        uint8 approvalCount;
        RestitutionStatus status;
    }

    mapping(uint256 => Payment) public payments;
    mapping(uint256 => RestitutionRequest) public restitutions;
    mapping(address => Role) public userRole;
    mapping(string => uint256) public npwpToPaymentCount;

    uint256 public paymentCounter;
    uint256 public restitutionCounter;

    event PaymentRecorded(
        uint256 indexed paymentId,
        address indexed wpAddress,
        string npwp,
        uint256 amount,
        string allocationCode
    );

    event RestitutionRequested(
        uint256 indexed requestId,
        uint256 indexed paymentId,
        uint256 amount
    );
```

File Edit Selection View Go Run Terminal Help Teknologi Blockchain

TaxTransparency.sol 1

TaxTransparency.sol > TaxTransparency

```
contract TaxTransparency {
    event RestitutionApproved(
        uint256 indexed requestId,
        uint8 approvalCount
    );

    modifier onlyVerifiedWP() {
        require(userRole[msg.sender] == Role.WP, "Not verified WP");
        _;
    }

    modifier onlyAuthorizedVerifier() {
        require(
            userRole[msg.sender] == Role.AR ||
            userRole[msg.sender] == Role.VERIFIER ||
            userRole[msg.sender] == Role.KPP_HEAD,
            "Not authorized"
        );
        _;
    }

    function recordPayment(
        string memory _npwp,
        string memory _taxType,
        uint256 _amount,
        string memory _documentHash,
        string memory _allocationCode
    ) public onlyVerifiedWP returns (uint256) {
        paymentCounter++;

        payments[paymentCounter] = Payment({
            paymentId: paymentCounter,
            npwp: _npwp,
            taxType: _taxType,
            amount: _amount,
            timestamp: block.timestamp,
            wpAddress: msg.sender,
            documentHash: _documentHash,
            allocationCode: _allocationCode,
            status: PaymentStatus.POSTED
        });

        npwpToPaymentCount[_npwp]++;

        emit PaymentRecorded(
            paymentCounter,
            msg.sender,
            _npwp,
            _amount,
            _allocationCode
        );

        return paymentCounter;
    }
```

File Edit Selection View Go Run Terminal Help Teknologi Blockchain

TaxTransparency.sol 1

TaxTransparency.sol > TaxTransparency

```
contract TaxTransparency {

    function requestRestitution(
        uint256 _paymentId,
        uint256 _amount,
        string memory _reason
    ) public onlyVerifiedWP returns (uint256) {
        require(_paymentId <= paymentCounter, "Payment not found");
        require(_amount <= payments[_paymentId].amount, "Amount exceeds payment");
        require(payments[_paymentId].wpAddress == msg.sender, "Not owner");

        restitutionCounter++;

        restitutions[restitutionCounter].requestId = restitutionCounter;
        restitutions[restitutionCounter].paymentId = _paymentId;
        restitutions[restitutionCounter].amount = _amount;
        restitutions[restitutionCounter].requester = msg.sender;
        restitutions[restitutionCounter].requestDate = block.timestamp;
        restitutions[restitutionCounter].status = RestitutionStatus.PENDING;

        emit RestitutionRequested(
            restitutionCounter,
            _paymentId,
            _amount
        );

        return restitutionCounter;
    }

    function approveRestitution(
        uint256 _requestId,
        uint8 _approvalIndex
    ) public onlyAuthorizedVerifier returns (bool) {
        require(_requestId <= restitutionCounter, "Request not found");
        require(_approvalIndex < 3, "Invalid approval index");
        require(
            restitutions[_requestId].status == RestitutionStatus.PENDING,
            "Request not pending"
        );

        RestitutionRequest storage request = restitutions[_requestId];

        require(!request.approvals[_approvalIndex], "Already approved");

        request.approvals[_approvalIndex] = true;
        request.approvalCount++;

        if (request.approvalCount >= 2) {
            request.status = RestitutionStatus.APPROVED;
            emit RestitutionApproved(_requestId, request.approvalCount);
        }

        return true;
    }

```

File Edit Selection View Go Run Terminal Help Teknologi Blockchain

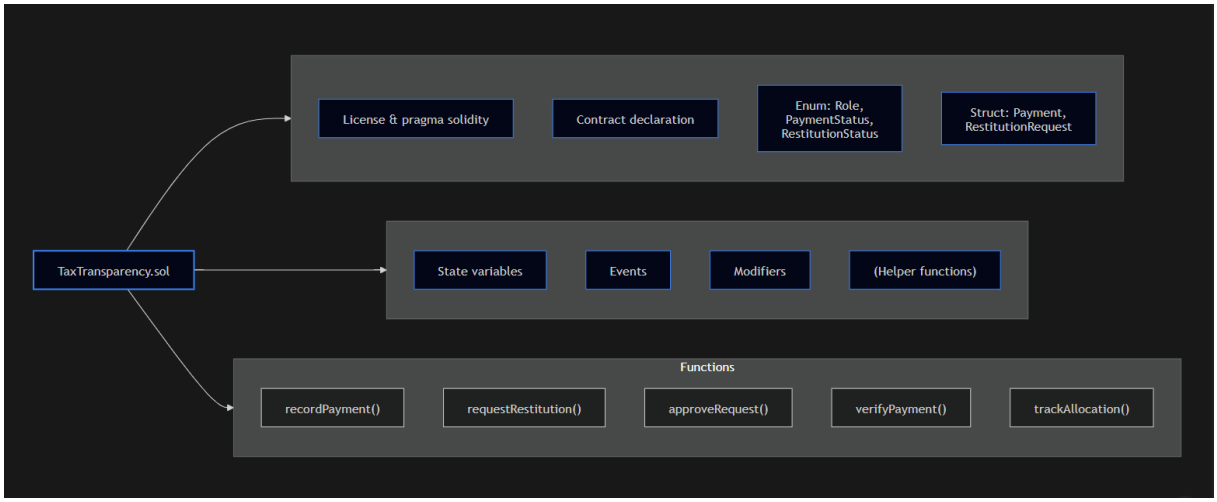
TaxTransparency.sol 1

TaxTransparency.sol > TaxTransparency

```
contract TaxTransparency {
    function verifyPayment(uint256 _paymentId)
        public
        view
        returns (
            string memory npwp,
            string memory taxType,
            uint256 amount,
            string memory status,
            string memory allocationCode,
            uint256 timestamp
        )
    {
        require(_paymentId <= paymentCounter, "Payment not found");

        Payment memory payment = payments[_paymentId];

        return (
            payment.npwp,
            payment.taxType,
            payment.amount,
            paymentStatusToString(payment.status),
            payment.allocationCode,
            payment.timestamp
        );
    }

    function paymentStatusToString(PaymentStatus status)
        internal
        pure
        returns (string memory)
    {
        if (status == PaymentStatus.PENDING) return "PENDING";
        if (status == PaymentStatus.POSTED) return "POSTED";
        if (status == PaymentStatus.ALLOCATED) return "ALLOCATED";
        if (status == PaymentStatus.RESTITUTED) return "RESTITUTED";
        return "UNKNOWN";
```

Contract Address : 0xd9145CCE52D386f254917e481eB44e9943F39138

Diagram Struktur Kode dalam TaxTransparency.sol :



Link Repository Github :

https://github.com/ahmadrajif10/UTS-Teknologi-BlockChain-Kelompok-10.git